

Study Note: Software testing Tutorial (1-10)

Study Note

[Software Testing Tutorial #1 - What is Software Testing | With Examples](#)



What is Software Testing?

1. Definition & Core Concept

- **Quality Assurance in General:** In everyday life, quality assurance is vital for any product (such as cars or electronic appliances) or service. Companies ensure they deliver quality to the market through a defined, predefined testing process.
- **Software Testing:** Software refers to any application (such as websites or mobile apps) built with programming languages to fulfill business logic. Applying the testing process to these applications—to verify that they function properly before reaching consumers—is known as software testing.

2. Practical Examples

- **Mobile Applications (e.g., WhatsApp):** Testers verify specific individual features and functionalities, such as sending text messages, making calls, adding contacts, and updating profile pictures.
- **E-Commerce Websites:** Testers verify key operational workflows, such as adding items to a shopping cart, removing items from a cart, and completing checkout procedures.

3. The Software Testing Life Cycle

Software testing is not a standalone activity; it is an ongoing life cycle that encompasses both static and dynamic approaches:

- **Static Approach:** Takes place during the analysis and design phase before executing the application code.
 - **Requirement Analysis:** Reviewing user stories or requirements (e.g., in Agile/Scrum) to spot ambiguities, missing details, or logic gaps.
 - **Test Case Design:** Writing step-by-step test cases to ensure complete requirement coverage.
- **Dynamic Approach:** Takes place when the application code is built and deployed.
 - **Test Execution:** Testers launch the software and physically perform the actions outlined in the test cases (e.g., clicking to add or remove an item).
 - **Evaluating Results:** Comparing the application's actual behavior against expected requirement criteria to determine if each test case Passes or Fails.

[Software Testing Tutorial #2 - What is Manual Testing | With Examples](#)



What is Manual Testing?

1. Definition & Core Concept

- **Manual Testing Defined:** Manual testing refers to the practice of testing software through human effort without relying on automated scripts, tools, or programming languages.
- **The Initial Testing Phase:** It is the primary and foundational testing phase conducted on any new application or feature to evaluate functionality.

2. Manual Testing vs. Automation Testing

Software testing is broadly divided into two main categories:

- **Manual Testing:** Executed manually by test engineers who interact with the application directly.
- **Automation Testing:** Uses specialized tools and scripts (such as Selenium for web applications) to execute test steps automatically.
- **Execution Sequence:** Manual testing is always performed first. Only after manual test cases pass properly and no critical defects remain are test cases considered for automation.

3. Methodology Agnostic Workflow

The fundamental phases of the manual testing life cycle remain identical regardless of the software development methodology being followed—whether Waterfall, V-Model, or Agile/Scrum.

4. Key Phases of the Manual Testing Life Cycle

The manual testing process follows a structured, step-by-step workflow:

1. **Requirement Analysis:** Testers review and analyze customer requirements or user stories (such as adding or removing items in an e-commerce shopping cart). The goal is to identify potential logic gaps or missing details and provide feedback to the business team.
2. **Test Planning:** Defining the overall testing strategy.
 - *Waterfall / V-Model:* Involves writing comprehensive master test plans for long-term projects.
 - *Agile / Scrum:* Involves creating concise, simple test plans tailored to short 2-to-4-week sprint iterations.
3. **Test Case Design:** Translating the analyzed requirements and scenarios into written, step-by-step test cases.
4. **Test Execution:** After developers deploy the code to a test environment, testers manually launch the application in supported web browsers (e.g., Chrome, Firefox) and perform the steps.
5. **Defect / Bug Logging & Retesting:** Any discrepancy between the expected result and actual behavior is logged as a defect or bug. It is assigned to a developer for fixing, after which the tester retests the fix and closes the defect if resolved.
6. **Test Closure & Reporting:** Summarizing test results. Formal test exit reports are written for extended Waterfall/V-Model projects, while short Agile/Scrum iterations require minimal, streamlined closure documentation.



Objectives of Software Testing

1. Overview

- **Role in the Learning Series:** This tutorial builds upon the fundamental concepts of software testing and manual testing by focusing specifically on why testing is performed.
- **Core Purpose:** The primary role of a testing team is structured around four key objectives to ensure the deliverable is defect-free, high-quality, requirement-aligned, and practical for end users.

2. The Four Key Objectives of Software Testing

Objective 1: Find Defects and Prevent Defects

Testing addresses defects through both dynamic and static testing approaches:

- **Finding Defects (Dynamic Approach):** When executing the software, testers actively look for functional failures.
 - *Example:* Launching an e-commerce website and attempting to add an item to the shopping cart; if the item fails to be added, a defect has been discovered.
- **Preventing Defects (Static Approach):** Before development code is written, testers review requirement documentation and collaborate with business analysts and developers.
 - *Process:* By analyzing requirements early, identifying documentation gaps, and writing test cases prior to implementation, testers prevent defects from being coded in the first place.

Objective 2: Ensure Good Quality of the Product

- **Early Bug Discovery:** Catching and fixing critical defects early in the lifecycle inherently improves software stability and quality.
- **Holistic User Experience:** Quality is not restricted strictly to functional code; testers also evaluate non-functional aspects such as look-and-feel, user interface (UI) presentation, and overall navigation.

Objective 3: Determine that the Software Meets Product Requirements

- **Aligning with Business Goals:** Software requirements are defined by the business or client.
 - *Example:* A banking client requests a portal that supports Chrome and Firefox, featuring a homepage "Register" button that opens a customer registration form.
- **Verification & Retesting:** Testing confirms that every functional requirement specified by the client is properly implemented in the application. If a required feature fails, a defect is logged, fixed by developers, and retested until requirement compliance is achieved.

Objective 4: Ensure the Software is "Fit for Use"

- **Beyond Literal Requirements:** Meeting client requirements on paper does not automatically

guarantee that a product is practical or usable in real-world scenarios.

- **The Car Analogy:** If a client requests a car with four wheels, a manufacturer could attach four wheels onto the roof or sides. While technically meeting the literal requirement ("a car structure with four wheels"), the product is completely unusable and not fit for driving.
- **Practical Usability:** Testers bridge the gap between written requirements and practical usability, ensuring the final application operates logically and effectively for the end user.

[Software Testing Tutorial #4 - Importance of Software Testing](#)



Importance of Software Testing

1. Overview & Core Purpose

- **Core Purpose:** The primary goal of software testing is to detect and resolve critical defects upfront in the software delivery cycle, preventing operational, security, and financial catastrophes.

2. Real-World Case Studies of Software Failures

Even major technology giants with large QA teams can face catastrophic software failures if critical bugs or capacity bottlenecks leak into production:

- **Flipkart (October 2014 – "Big Billion Day" Sale):**
 - *Issue:* Extreme user traffic overwhelmed the e-commerce platform's infrastructure during a major promotional sale.
 - *Impact:* Led to widespread order cancellations, processing delays, and significant harm to the company's brand reputation.
- **Yahoo (September 2016 – Data Security Breach):**
 - *Issue:* A major security flaw within the software environment resulted in a massive security breach.
 - *Impact:* Approximately 500 million user credentials were compromised, severely damaging user trust and corporate reputation.
- **Royal Bank of Scotland / RBS (June 2015 – Payment Processing Failure):**
 - *Issue:* A technical glitch in the banking software prevented the system from processing 600,000 (6 lakh) customer payments.
 - *Impact:* RBS was hit with a £56 million regulatory fine alongside direct financial losses and reputational damage.

3. Major Consequences of Uncaught Software Bugs

Failing to catch critical defects prior to release results in multi-dimensional damage to an organization:

- **Financial Losses:** Defects cause direct financial damage through lost transactions, cancelled

- Financial Losses: Defects cause direct financial damage through lost transactions, cancelled orders, costly emergency fixes, and heavy regulatory fines.
- Reputational Damage: Outages and software glitches erode customer trust and severely harm company reputation.
- Security & Data Breaches: System vulnerabilities allow sensitive user credentials and confidential data to be compromised.
- Safety & Life-Threatening Risks: In safety-critical software domains, unaddressed software bugs can pose life-threatening hazards.

4. Key Takeaways: Why Software Testing is Essential

Software testing is essential to guaranteeing business continuity and product reliability:

- Risk Reduction: Proactively identifies and mitigates operational and business risks throughout development.
- Finding Bugs Upfront: Discovers and fixes bugs early in the lifecycle rather than waiting for issues to surface in production.
- Improving Software Quality: Drives overall product stability, performance, and user satisfaction.
- Protecting Revenue & Reputation: Safeguards the organization against financial penalties, lost revenue, and brand erosion.

[Software Testing Tutorial #5 - Seven Principles of Software Testing](#)



Seven Principles of Software Testing

1. Overview & Core Importance

- **Role in Software Testing:** Understanding the foundational principles of software testing is essential for applying effective strategies in real-world scenarios and answering key software QA interview questions¹.
- **Core Purpose:** These seven principles serve as guiding rules for test engineers to plan, design, and execute testing activities efficiently across various project domains¹.

2. The Seven Principles of Software Testing

Principle 1: Early Testing

- **Start Early in the SDLC:** Testing activities should begin as early as possible in the Software Development Life Cycle (SDLC)¹.
- **Evolution Beyond Waterfall:** In traditional Waterfall models, testing was conducted late after development was complete². Modern iterative frameworks like **Agile/Scrum** involve testing in every short cycle or iteration, embedding QA into all phases (from requirement analysis onwards)².

Principle 2: Testing Shows the Presence of Defects

- **Defects vs. Defect-Free Guarantee:** Testing can prove that defects are present in the software, but it **can never prove that software is 100% defect-free**²³.
- **Real-World Production Realities:** Even major organizations with massive QA capabilities (such as Yahoo or Flipkart) have experienced live production failures³. Differences between test and production environments (such as hardware or network variations) mean unexpected defects can still surface in production³.

Principle 3: Exhaustive Testing is Impossible

- **Combinatorial Explosion:** Testing every single combination of inputs, permutations, and preconditions across an application is impossible⁴.
- **Impracticality:** Even simple web forms with a few text fields (e.g., name, date of birth) yield an endless number of input combinations, making 100% test coverage unachievable—even with automation tools⁴.
- **Risk-Based Focus:** Because exhaustive testing is impossible, testing efforts must focus on risks, priorities, and high-impact areas⁴.

Principle 4: Defect Clustering

- **Concentration in Complex Modules:** A small number of modules usually contain the majority of defects discovered in a software system⁴⁵.
- **Root Causes:** Defect clustering occurs due to high module complexity, intricate business logic, or complex integrations⁵.
- **Targeted Focus:** If a tester identifies multiple bugs in a specific module, it is highly likely that more hidden defects exist within that same module⁵.

Principle 5: Pesticide Paradox

- **Immunity to Repeated Tests:** Executing the exact same set of test cases repeatedly will eventually stop discovering new defects, similar to pests building immunity against repeated applications of a single pesticide⁶.
- **Continuous Revision:** To uncover new bugs, test cases must be regularly reviewed, updated, revised, and expanded with new test scenarios⁶.

Principle 6: Testing is Context Dependent

- **Tailored Testing Strategies:** Software testing is executed differently depending on the domain, type, and criticality of the application⁶⁷.
- **Domain Contrast:**
 - *Personal Blog:* Requires minimal testing since downtime or minor glitches have very little business impact⁷.
 - *Banking Application or Aviation Software:* Requires rigorous testing, strict security verification, and compliance documentation because failures risk severe financial loss, brand damage, or life-threatening hazards⁷⁸.

Principle 7: Absence-of-Errors Fallacy

- **Defect-Free → Fit for Use:** Finding and fixing all bugs does not automatically mean the software is successful or usable⁸⁹.
- **The "Fit for Use" Requirement:** Even if software meets written requirements line-by-line without any bugs, it is a failure if it fails to fulfill real user needs or practical business workflows (e.g., building a car with four wheels attached to the wrong locations meets the literal requirement of having four wheels, but remains unusable)



Software Development Life Cycle (SDLC)

1. Overview & Definition

- **Software Development Life Cycle (SDLC):** A structured framework followed by software organizations to plan, design, build, test, deploy, and maintain high-quality software systems.
- **Core Purpose:** Provides a clear, phase-by-phase workflow to ensure that software applications are delivered efficiently, meet business requirements, and remain stable throughout their operational lifecycle.

2. Key Phases of the Software Development Life Cycle

Phase 1: Requirement Gathering & Analysis

- **Collecting Requirements:** Business analysts and software teams collect functional and non-functional requirements directly from the customer.
- **Clarifying Logic:** Requirement details are thoroughly reviewed and clarified with the client to eliminate ambiguities before technical work begins¹.

Phase 2: Design Phase

- **Architectural Blueprint:** Technical architects define the overall technical structure and visual layout of the software¹.
- **Two Core Sub-Phases:**
 - **High-Level Design (HLD):** Defines the macro-level architecture, main system modules, and system relationships¹.
 - **Low-Level Design (LLD):** Defines granular module details, logical flows, and database schemas¹.
- **Customer Alignment:** Visual layouts and architecture diagrams are presented to the customer to confirm functional placement before writing code¹.

Phase 3: Development / Coding Phase

- **Constructing the Software:** Once design blueprints are finalized, developers begin the actual programming work¹.
- **Code Implementation:** Just as builders construct a house by laying bricks over a foundation, developers build application features by translating design specifications into source code¹.

Phase 4: Testing Phase

- **Verification & Validation:** Quality assurance engineers test the compiled code against the customer's specified requirements.
- **Defect Resolution:** Testers execute test cases, identify functional or non-functional bugs, and assign them to developers to fix before release.

Phase 5: Deployment Phase

- **Production Release:** After testing is completed and key defects are resolved, the application is deployed to production servers for live user access.

Phase 6: Maintenance Phase

- **Ongoing System Maintenance:** Continuously maintaining live applications to prevent performance bottlenecks, system crashes, or hardware compatibility issues².

- **Production Defect Fixes:** Tracking, fixing, and redeploying patches for production issues reported by real customers².
- **Connection to Testing Principles:** Reaffirms the principle that achieving 100% defect-free software is impossible; continuous maintenance safeguards production stability when post-release bugs surface².

3. Sequential Workflow Summary

Requirement → Analysis → Design → Development → Testing → Deployment → Maintenance

[Software Testing Tutorial #7 - Software Testing Life Cycle \(STLC\)](#)



Software Testing Life Cycle (STLC)

1. Overview & Definition

- **Definition of STLC:** The Software Testing Life Cycle (STLC) is a sequence of specific phases and processes executed by a quality assurance team to verify that a software application meets business requirements and quality standards.
- **Relation to SDLC:** STLC operates within the broader Software Development Life Cycle (SDLC). While SDLC encompasses overall software development (from requirements to deployment and maintenance), STLC focuses specifically on quality verification and testing activities.
- **Methodology Flexibility:** The core phases of STLC remain consistent whether a project follows Waterfall, V-Model, or Agile/Scrum development models.

2. The Seven Phases of the Software Testing Life Cycle

Phase 1: Requirement Analysis

- **Analyzing Requirements:** Testers thoroughly review customer requirement documents or Agile user stories to understand intended features and functionality.
- **Identifying Gaps:** Testers identify missing logic, ambiguities, or inconsistencies in requirements and collaborate with Business Analysts (BAs) or clients to resolve them early.
- **Determining Test Scope:** The team analyzes what functionalities need to be tested and identifies potential test scenarios.

Phase 2: Test Planning

- **Defining Strategy & Scope:** Test leads or QA engineers define the overall testing scope, out-of-scope items, timelines, resources, and entry/exit criteria.
- **Waterfall vs. Agile Planning:**
 - *Waterfall:* Involves creating comprehensive, formal Master and Phase Test Plans for long project timelines.
 - *Agile/Scrum:* Involves creating a concise 1-to-2 page test plan tailored specifically to the

- *Agile/Scrum* involves creating a concise, 1 to 2 page test plan tailored specifically to the user stories selected for a short 2-to-4 week sprint.

Phase 3: Test Case Design & Development

- Applying Test Design Techniques: Testers apply formal design techniques—such as Equivalence Partitioning and Boundary Value Analysis—to ensure thorough test coverage.
- Authoring Test Cases: Detailed test cases and scenarios are written, organized, and uploaded into test management tools categorized by feature or requirement.

Phase 4: Test Environment Setup

- Preparing Infrastructure: Establishing the hardware, software, database, and network configurations required to deploy and run the application.
- Environment Management: Depending on complexity (e.g., in Banking or Telecom sectors), the environment may be managed by a dedicated infrastructure team or by the QA team itself.
- Test Data Preparation: Testers prepare and furnish the necessary test data required for executing test cases.

Phase 5: Test Execution

- Running Test Cases: Once application code is deployed to the test environment, testers launch the application (e.g., in a web browser) and manually execute written test cases step-by-step.
- Defect Logging & Retesting: Actual application behavior is compared against expected results. Discrepancies are logged as defects, assigned to developers for fixes, and retested once resolved.

Phase 6: Test Reporting

- Generating Test Summary Reports: Producing formal documentation and metrics that summarize test execution status, pass/fail ratios, open defect counts, and overall system stability.
- Agile Automation: In Agile environments, execution reporting is often automated via tools like Jira gadgets and dashboards throughout the sprint.

Phase 7: Test Closure

- Final Sign-off & Handoff: Conducted after testing is complete and the software is ready for production deployment.
- Archiving Test Artifacts: QA teams compile final test summary reports, test case documentation, and execution artifacts to share with client or maintenance teams.
- Waterfall vs. Agile Closure:
 - *Waterfall*: Conducted at the end of the entire project lifecycle.
 - *Agile/Scrum*: Conducted per major release (e.g., every 1 to 3 months) rather than at the end of every individual 2-week sprint.

3. Comparison Summary: STLC in Waterfall vs. Agile

Activity / Phase	Waterfall Model	Agile / Scrum Framework
Project Timeline	Extended duration (months to a year).	Fast-paced, short 2-to-4 week sprints.
Test Plan	Comprehensive, lengthy document covering all project phases.	Concise 1 to 2 page sprint-specific test plan.

Test Plan Scope	Comprehensive, lengthy Master Test Plan.	Concise 1-to-2 page sprint test plan.
Collaboration	Sequential handoffs between phases.	Continuous daily communication between QA, BAs, and developers.
Test Reporting	Manual creation of formal test reports.	Automated real-time dashboards (e.g., Jira gadgets).
Test Closure	Performed once at project completion.	Performed at major release milestones (every 1–3 months).

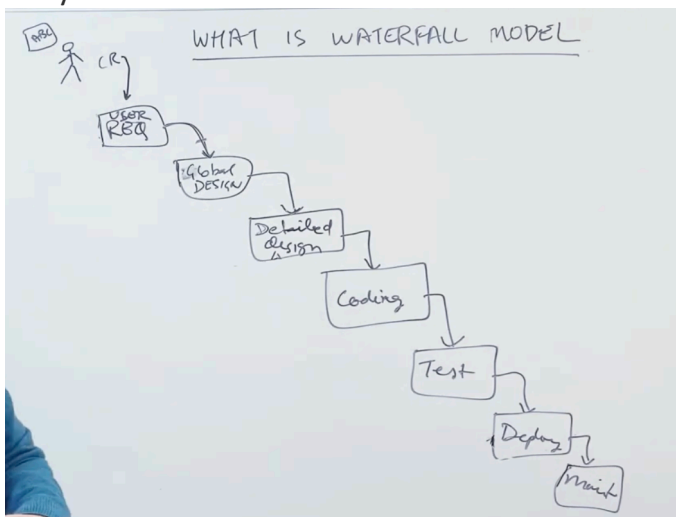
[Software Testing Tutorial #8 - Waterfall Model in Software Engineering](#)



Waterfall Model in Software Engineering

1. Overview & Definition

- **Sequential Flow:** The Waterfall Model is a traditional, linear sequential software development life cycle framework where project activities progress strictly downwards like a waterfall.
- **Extended Delivery Cycle:** In a traditional Waterfall approach, the complete application is planned, built, tested, and delivered in a single release cycle that often spans 6 months to 2 years.



2. Key Phases of the Waterfall Model

The Waterfall approach moves sequentially through seven distinct phases:

1. **User Requirements / Requirement Analysis:** Gathering, defining, and documenting all customer requirements and business expectations upfront before technical work starts.
2. **Global / High-Level Design (HLD):** Defining the macro-level system architecture, main system

modules, and integration points.

3. Detailed / Low-Level Design (LLD): Designing granular module specifications, logical workflows, and database schemas.
4. Coding / Development: Translating design specifications into functional source code.
5. Testing: QA engineers execute test cases to verify software functionality against requirements after coding is completed.
6. Deployment: Deploying the tested software to the live production environment for end-user access.
7. Maintenance: Ongoing post-release support, hardware compatibility maintenance, and patch deployments.

3. Key Limitations & Disadvantages (Cons)

- **Late Testing Phase:** Testing occurs late in the Software Development Life Cycle (SDLC) only after development is finished.
- **Deployment Delays:** Defect fixes discovered during late testing require rework and re-testing, causing severe delays in product deployment.
- **High Cost of Changes:** Design modifications or requirement changes triggered by late bug fixes increase development costs and further delay delivery timelines.
- **Inflexible Structure:** Accommodating changing business needs is extremely difficult once initial phases are closed.

[Software Testing Tutorial #9 - V Model in Software Engineering](#)



V Model in Software Engineering

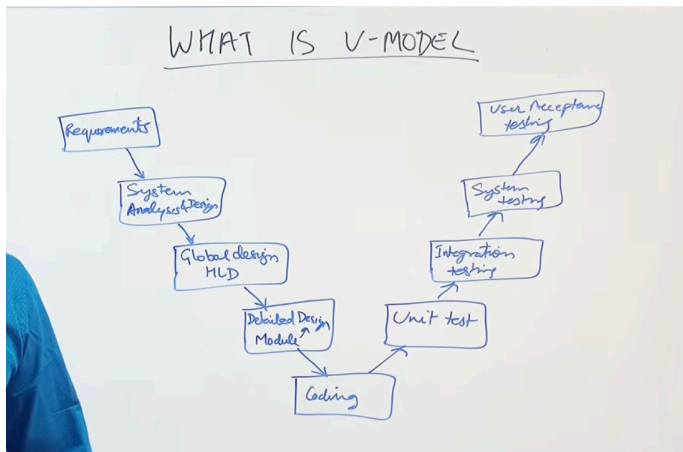
1. Overview & Evolution from Waterfall

- **Origin & Background:** The V-Model was introduced to address major limitations inherent in the traditional Waterfall software development model¹².
- **The Core Problem in Waterfall:** In the Waterfall approach, testing activities were deferred until late in the development life cycle, resulting in delayed feedback and late defect discovery¹².
- **Evolution:** The V-Model (Verification and Validation Model) overcomes these issues by involving the testing team continuously across every phase of software development¹².

2. Core Structure & Concept

- **The "V" Shape:** Unlike the linear cascade of the Waterfall model, the V-Model is structured in a V shape¹.
 - **Left Arm (Verification / Design Phases):** Focuses on requirement gathering, system analysis, global architectural design, detailed module design, and parallel test case authoring³more horiz.

- **Bottom (Coding Phase):** Represents the actual implementation where developers write source code based on module designs⁶⁷.
- **Right Arm (Validation / Testing Phases):** Focuses on executing test suites (Unit, Integration, System, and Acceptance testing) from the ground up⁷⁸.
- **Parallel Test Design:** A core principle of the V-Model is parallel test design, where test cases for each testing phase are authored simultaneously during their corresponding development phase³more_horiz.



3. Sequential Mapping of Phases (Design <-> Testing)

Phase 1: Requirement Analysis <-> User Acceptance Testing (UAT)

- **Design Action:** Customer requirements and user stories are gathered and analyzed³.
- **Testing Action:** The testing team immediately reviews these requirements to draft **User Acceptance Test (UAT) cases**³.
- **Benefit:** Drafting test cases during requirements analysis allows testers to catch documentation gaps and logic ambiguities early³.

Phase 2: System Analysis & Design <-> System Testing

- **Design Action:** System analysis evaluates the high-level application scope and its interactions across multiple internal or external systems (such as a banking card origination system interacting with backend processing and card embossing systems)⁴.
- **Testing Action:** Testers review system-level requirements to write **System Test Cases**⁴⁵.

Phase 3: Global / High-Level Design (HLD) <-> Integration Testing

- **Design Action:** Technical architects define the core application architecture and global design for how modules interact⁵.
- **Testing Action:** The testing team creates **Integration Test Cases** to verify data exchange between interconnected modules⁵⁷.

Phase 4: Detailed / Low-Level Design (LLD) <-> Unit Testing

- **Design Action:** Software architecture is broken down into detailed, granular module specifications⁶.
- **Testing Action:** Development or white-box test engineers review module specifications and write **Unit Test Cases**⁶.

Phase 5: Coding

- **Implementation:** Developers translate detailed module designs into functional source code67/.

Validation & Execution Flow (Bottom to Top)

1. **Unit Testing:** Individual code modules (such as a card application input form) are tested in isolation67.
2. **Integration Testing:** Testing is executed across multiple connected modules (such as verifying data flow between the frontend UI and backend database)78.
3. **System Testing:** The complete application is tested end-to-end alongside third-party integrations to verify system behavior8.
4. **User Acceptance Testing (UAT):** Business stakeholders or end customers execute UAT test cases (prepared during requirement analysis) to confirm the software meets business goals and is fit for use8.

4. Key Advantages of the V-Model

- **Early Defect Identification:** Involving testers from the requirement phase enables early defect identification, which significantly reduces fix costs compared to late discovery23.
- **Continuous QA Involvement:** Testing teams work actively alongside developers throughout the life cycle rather than waiting idly for code completion23.
- **Clear Traceability:** Every phase on the development side directly aligns with a corresponding testing phase on the validation side3more_horiz.

[Software Testing Tutorial #10 - Iterative Model in Software Engineering](#)



Iterative Model in Software Engineering

1. Overview & Background

- **Limitations of Previous Models:** Both the Waterfall model and the V-Model suffered from significant drawbacks related to lengthy delivery timelines.
- **The Long Duration Problem:** Although the V-Model successfully integrated testing into every phase of development, the total duration required to deliver functional value to customers remained long (often 6 months to 2 years).
- **Modern Market Demands:** In today's dynamic market conditions, organizations cannot afford to wait years to release software. The Iterative Development Model was introduced to solve this delayed delivery problem by enabling rapid and continuous releases.

2. Core Concept & Mechanism

- **Definition of an Iteration:** An iteration is a short, fixed-duration cycle (typically lasting 4 to 8 weeks, or 1 to 2 months).
- **Miniature Waterfall Cycles:** Within each iteration, a complete mini-lifecycle—comprising requirements analysis, design, development, testing, and deployment—is executed.

- **Incremental Release Process:** Instead of delivering the entire application in a single big release, the software is built and released in progressive phases (Phase 1, Phase 2, Phase 3, etc.):
 - *Iteration 1:* The team selects a foundational subset of features (e.g., initial application skeleton or basic login functionality) and carries them through requirements, design, development, testing, and deployment.
 - *Iteration 2 & Beyond:* The team picks the next set of features and repeats the exact same lifecycle phases, continuously building upon the existing application until all customer requirements are fulfilled.

3. Comparison: Waterfall / V-Model vs. Iterative Model

Aspect	Waterfall / V-Model	Iterative Model
Delivery Timeline	Extended release cycle (typically 6 months to 2 years).	Short, frequent release cycles (4 to 8 weeks / 1 to 2 months).
Release Strategy	Single delivery at the end of the entire project lifecycle.	Multi-phase releases delivering incremental feature additions.
Lifecycle Structure	One linear, large-scale pass through development phases.	Repeating mini-waterfall passes across short, sequential iterations.

4. Key Advantages & Relation to Agile

- **Rapid Value Delivery:** Enables organizations to deliver working software and business value to customers much faster.
- **Progressive Feature Expansion:** Allows software to be built, tested, and expanded incrementally, reducing the risk of large-scale delivery failures.
- **Stepping Stone to Agile:** By shortening development cycles and emphasizing continuous iteration, the Iterative Model serves as a conceptual foundation for modern Agile frameworks like Scrum.